Screen hat

Jeremiah Lowe, DC34

V1 System architecture

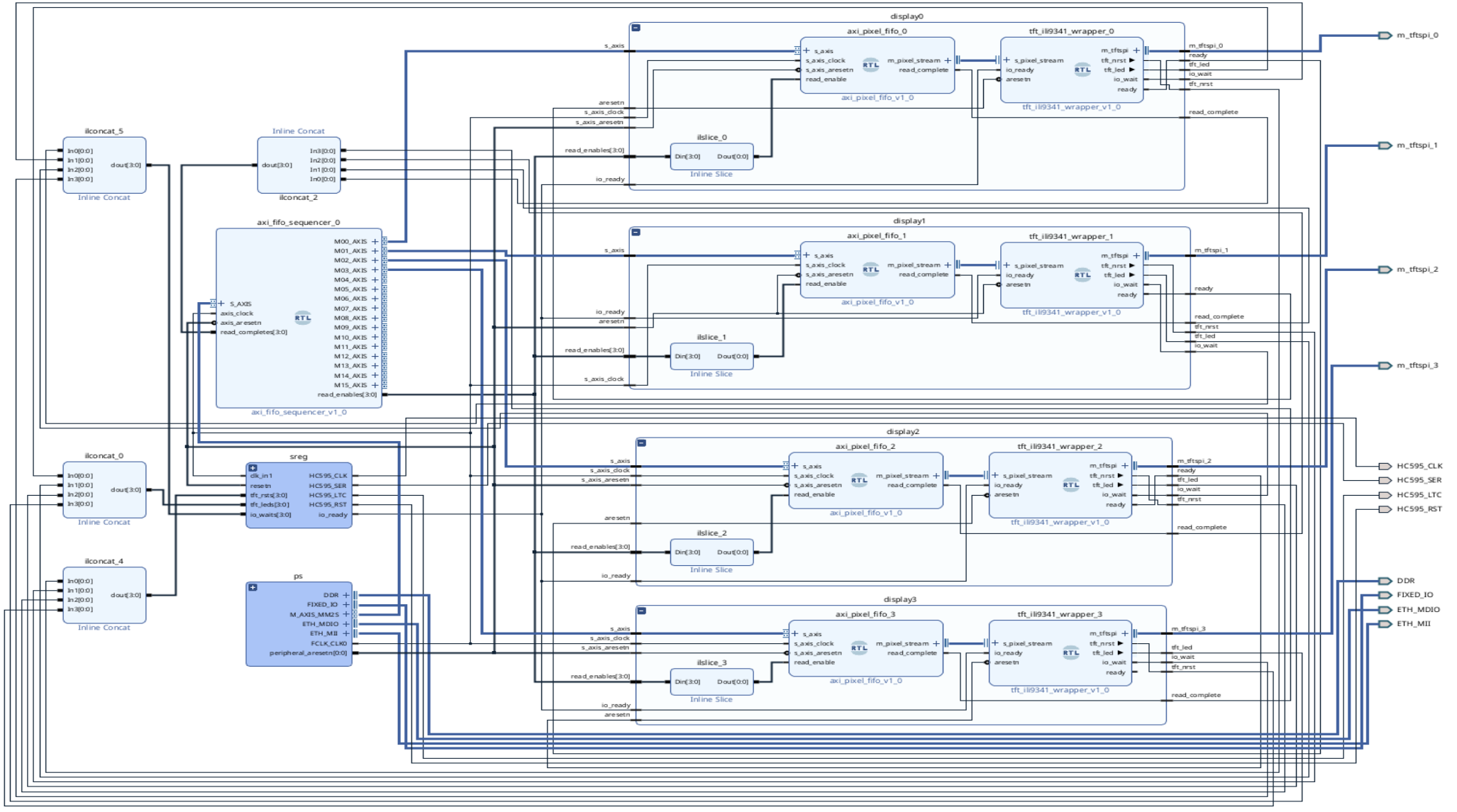
- Zynq-7010 based development board
  - Microphase Z7 lite, at the time $70, no BSP or timing information at all provided, just a markdown document with wiring
- Custom GPIO expander PCB
  - 14 displays, 6 pins each, 84 IO pins needed, board only has ~60
  - Dual channel 24-bit 595 based IO expanders added, PL driven
  - Converts 6 IO lines into 48, very slow, used for BL / RST lines
  - Ultra-efficient 3.3V switching regulator for all displays
- 240x320 16-Bit color ILI9341 SPI displays, bought around 30 for $5/piece
- USB powered, uses around 3W, surprisingly only 750mW for displays

System development

- Hand wired in an afternoon using 28AWG stranded wire, 100 mil standard pitch dupont connectors
- Most high speed (clock / data) lines were length matched-ish
- 330 ohm resistors added for impedence matching on clock and chip-select
- Small 4-display test bed is where most development was performed, Easier and less delicate to work with, smaller PL so quicker Vivado synthesis
- Vivado builds take 15 minutes for the 4 display version, 45 minutes for the complete version (granted there are is some extra debugging functionality I added)
- Handmade test PCBs for the GPIO expanders and power regulation since I didn’t have time to make a V2 PCB due to not ordering them soon enough.
- Dev board was advertised with 1Gbps PS ethernet, came with 100Mbps PL ethernet, had to add a Ethernet lite block with around 15 pins for MII interface

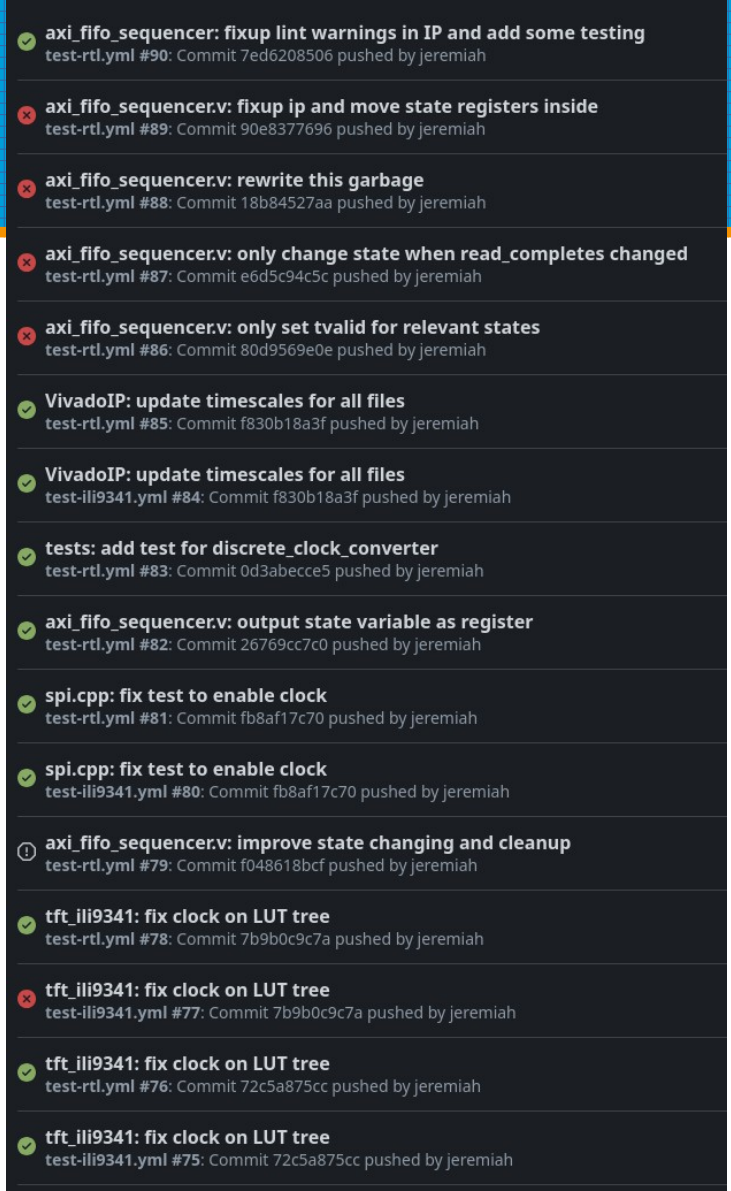
PL Architecture (cont)

- Displays are entirely PL driven, running at 60MHz
  - AXI DMA Generates an AXI4-Stream direct from PS DDR
  - AXI FIFO Sequencer splits the stream into 14 channels using a custom IP block, using counters to send one horizontal line per display
  - AXI Pixel FIFO buffers a horizontal line of pixels and converts it into a pixel stream, buffering is critical since SPI is over 18X slower then AXI, both run at 60MHz (the displays maximum clock is 10MHz, but you can run them up to 80MHz with the termination resistors, 60MHz seemed like a good compromise)
  - ILI9341 Driver block takes in a “pixel stream” and drives the SPI per display
  - Because of the sequencing all displays are updated in parallel helping with screen tearing and artifacting, Block RAM contraints prevent a PL based framebuffer
- Linux just has to load the FPGA and read/writes to a memory block, thats it!

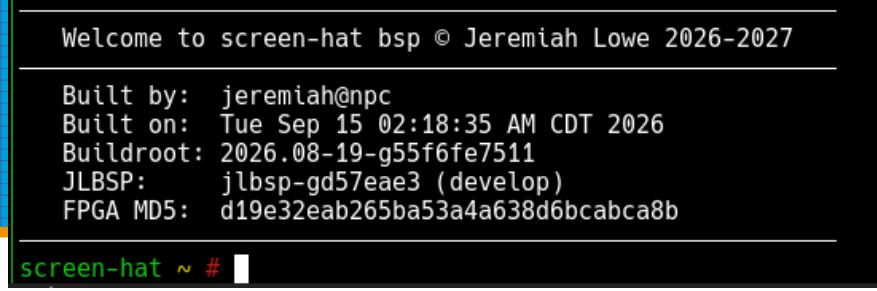

display0
axi_pixel_fifo_0
axi_pixel_fifo_v1_0
tft_ili9341_wrapper_0
tft_ili9341_wrapper_v1_0
ilslice_0
Inline Slice
display1
axi_pixel_fifo_1
tft_ili9341_wrapper_1
ilslice_1
display2
axi_pixel_fifo_2
tft_ili9341_wrapper_2
ilslice_2
display3
axi_pixel_fifo_3
tft_ili9341_wrapper_3
ilslice_3
ilconcat_5
Inline Concat
ilconcat_2
ilconcat_0
ilconcat_4
axi_fifo_sequencer_0
axi_fifo_sequencer_v1_0
sreg
ps
m_tftspi_0
m_tftspi_1
m_tftspi_2
m_tftspi_3
HC595_CLK
HC595_SER
HC595_LTC
HC595_RST
DDR
FIXED_IO
ETH_MDIO
ETH_MII

PL Development

- All of my IP is written in Verilog
- Compiled into C++ using Verilator
- Unit tested via CI for instant feedback, running the tests at command line finishes in seconds
- Tests run in parallel, with some dedicated tests to make sure IP blocks connected together will work together
- No need to open Vivado and wait over 30 minutes for synthesis / implementation just to find a small mistake
- Zero FPGA experience going into this so this was the longest part of the project since my initial IP was trash



PS Architecture

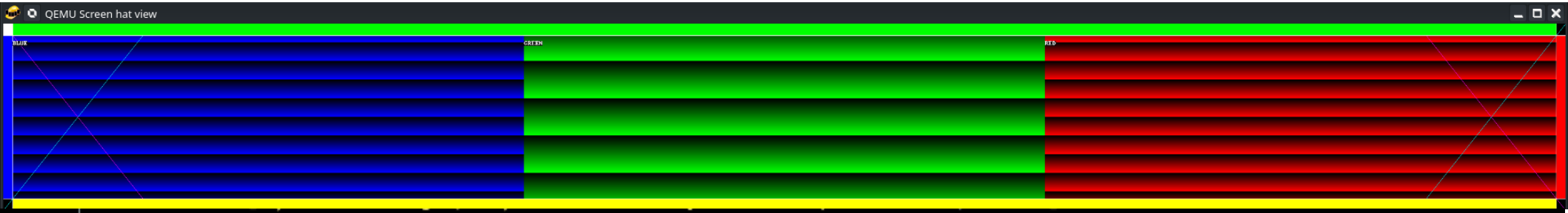


- Runs latest Xilinx uboot/linux, buildroot 2026.08
- An AXI GPIO for each group of pins, one for resets, one for backlights, PS MIOs are too much of a pain to work with due to odd numbering
- Multi stage FPGA init
  - S01load-fpga loads the FPGAs and DTBO containing PL Ethernet and PL GPIOs
  - S02init-displays deasserts an IO_RESET pin allowing the displays to initialize and then waits for an IO_READY, once high, another DTBO containing the DMA block and graphics driver can be safely loaded. This was my “fix” to some initialization timing issues.
- Linux kernel framebuffer driver utilizing Linux DMA engine under the hood
  - The Linux DMA engine calls the AXI DMA driver, updating the screen
- Rust based graphics library and userspace app handles animations
- QEMU based emulation of the device driver utilizing /dev/shm and an SDL app

QEMU Emulation

- QEMU uses /dev/shm/screen-hat-mem, SDL2 viewer app reads and displays contents of /dev/shm/screen-hat-mem
- Entire memory of PS is mapped so only the display region is mmap'd in the SDL viewer app.
- Ideally would like to combine with verilator and simulate the entire PL stack as a whole (bit bang an AXI stream from /dev/shm region, pass into IP using verilator compiled PL logic, generate an image from the 14x SPI buses coming out, would be very CPU intensive)
- SDL viewer app is currently very simple, ~170 lines of C, just creates a texture from the mmap region, scales to window size and renders, some code to save/load window position/size as well as allow the window to be resizable
- FPGA loading is skipped if running in QEMU via 'qemu=1' passed in the bootargs, QEMU specific device tree enables the GEM ethernet so networking can be used inside the VM

QEMU emulation (cont)




```
qemu-system-arm \
    -M xilinx-zynq-a9 \
    -object "memory-backend-file,id=mem,size=512M,mem-path=${ramf},share=on" \
    -machine memory-backend=mem \
    -display none \
    -serial mon:stdio \
    -kernel "${BINARIES_DIR}/uImage" \
    -dtb "${BINARIES_DIR}/z7lite-qemu.dtb" \
    -append "console=ttyPS0,115200 root=/dev/mmcblk0p2 rw rootwait qemu=1" \
    -net nic -net user,id=net0,hostfwd=tcp::2222-:22 \
    -drive file="${qemu_img}",if=sd,format=raw
```

File: **board/z7lite/dts/xilinx/z7lite-qemu.dts**

```
/dts-v1/;

#define LINUX
#define QEMU

#include "zynq-z7lite.dtsi"

&gem0 {
    // Use the GEM since the PL ethernet will not be ac
    status = "okay";
    // And a fake PHY to go with it
    phy-mode = "rgmii-id";
    fixed-link {
        speed = <1000>;
        full-duplex;
    };
};

```

Recent improvements

- Fix driver so it works as a proper framebuffer device
  - Now supports full SDL2, X11, and console support
  - Adding QEMU to the project has helped massively with this
  - Kernel module can be scp’d over and tested via a single call to modprobe
  - Xorg does need some magic parameters to work in buildroot, this was because buildroot somehow breaks the automatic shared object detection / loading used by X11
  - Easily fixed in Xorg.conf

Future improvements

- Rewrite driver to be a DRM device instead of FB device
  - Not worth doing until I have an SoC with a GPU
- Version 2.0, based on the Avnet ZU3EG Ultrascale SOM
  - I got around 10 of these real cheap for $100/piece still sealed in box
  - Zynq 7000 32-bit ARM cores are really underpowered, especially for computer graphics due to lack of any real NEON support
  - Much larger PL plus Mali GPU will allow for hardware acceleration
  - Have them booting via a PCIe carrier board
  - Started the PCB design and have the connectors in

Github repos

- All of this is on Github, mostly GPLv2, some things MIT
- This powerpoint is also in the github repo
- https://github.com/abc123me/screen-hat-defcon